



MGM University
Jawaharlal Nehru Engineering College
Aurangabad (MH)

NAAC 'A' Grade, ISO 9001:2015, 14001:2015 Certified, AICTE Approved

Department of Mechanical Engineering
(Robotics and Artificial Intelligence)

Fundamentals of Artificial Intelligence Lab Manual

Prepared By: -
Prof Pankaj Dhoble
Faculty

Reviewed By
Dr M.S. Kadam
Head of Department

Approved by
Dr H.H. Shinde
Dean (Engineering and
Technology)

Vision and Mission of the Department

Vision of Mechanical Department

To establish the state of the art learning center in Mechanical Engineering which will impart global competence, enterprising skills, professional attitude and human values in the student.

Mission of Mechanical Department

1. To impart quality technical education to the students.
2. To develop comprehensive competence in the students through various modes of learning.
3. To enable students for higher studies and competitive examinations.
4. To facilitate students and industry professionals for continuous improvement and innovation.

Program Educational Objectives:

- [1] Use core competence acquired in various areas of Mechanical Engineering to solve techno-managerial issues for creating innovative products that lead to better livelihoods & economy of resources.
- [2] To establish themselves as effective collaborators and innovators to address technical, managerial and social challenges.
- [3] To equip students for their professional development through lifelong learning and career advancement along with organizational growth.
- [4] Serve as a driving force for proactive change in industry, society and nation.

PROGRAM SPECIFIC OUTCOMES

Student should have

- 1) An ability to work professionally in mechanical systems including design, analysis, production, measurement and quality control.
- 2) An ability to work on diverse disciplinary tasks including manufacturing, materials, thermal, automobile, robotics, mechatronics, engineering software tools, automation and computational fluid dynamics.

Lab Objective:

1. To introduce students the fundamentals of Artificial Intelligence its applications in various fields, their applications.
2. Understand evolution and different Use Cases and AI principles.
3. To familiarize students with different processes in Hardware Integration of AI.
4. Describe about mechanical properties and geometric issues relating to specific AI based projects.
5. Extend the knowledge of students for applications in Aerospace, defenses, automobile, Bio-Medical and General Engineering Industries.
6. Understand the terminologies related to re-engineering, forward engineering, and reverse engineering.

Lab Outcome: After the completion of this course students will be able to,

1. Understand the importance of Ethical AI.
2. Design Model an Electronic Circuits as well as Assembly.
3. Develop AI models.
4. Understand BFS, DFS.
5. Develop 3D Model
6. Measure parameters and calibrate there AI.

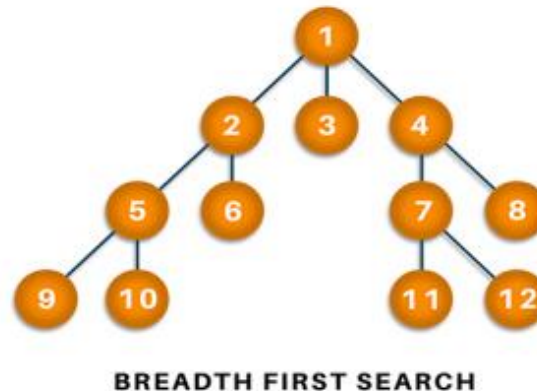
Mandatory instructions for students:

1. Inspect machinery before each use-ensuring that safeguarding systems are in place and working properly.
2. Follow all machine operating and maintenance instructions.
3. Use appropriate personal protective equipment-such as safety glasses or goggles, gloves and hearing protection.
4. Keep hands away from all moving parts-at all times.
5. Keep hands away from all moving parts-at all times.
6. Be on time.
7. Participate in class
8. Focus on the subject matter
9. Listen attentively and take notes
10. Practice what you have learned
11. Enjoy the class

Practical no. 1

Aim: Write a Program to Implement Breadth FirstSearch using Python

Theory:



Breadth-First Search (BFS) is an algorithm used for traversing graphs or trees. Traversing means visiting each node of the graph. Breadth-First Search is a recursive algorithm to search all the vertices of a graph or a tree. BFS in python can be implemented by using data structures like a dictionary and lists. Breadth-First Search in tree and graph is almost the same. The only difference is that the graph may contain cycles, so we may traverse to the same node again.

BFS Algorithm:

Before learning the python code for Breadth-First and its output, let us go through the algorithm it follows for the same. We can take the example of Rubik's Cube for the instance. Rubik's Cube is seen as searching for a path to convert it from a full mess of colors to a single color. So, comparing the Rubik's Cube to the graph, we can say that the possible state of the cube is corresponding to the nodes of the graph and the possible actions of the cube is corresponding to the edges of the graph.

As breadth-first search is the process of traversing each node of the graph, a standard BFS algorithm traverses each vertex of the graph into two parts: 1) Visited 2) Not Visited. So, the purpose of the algorithm is to visit all the vertex while avoiding cycles

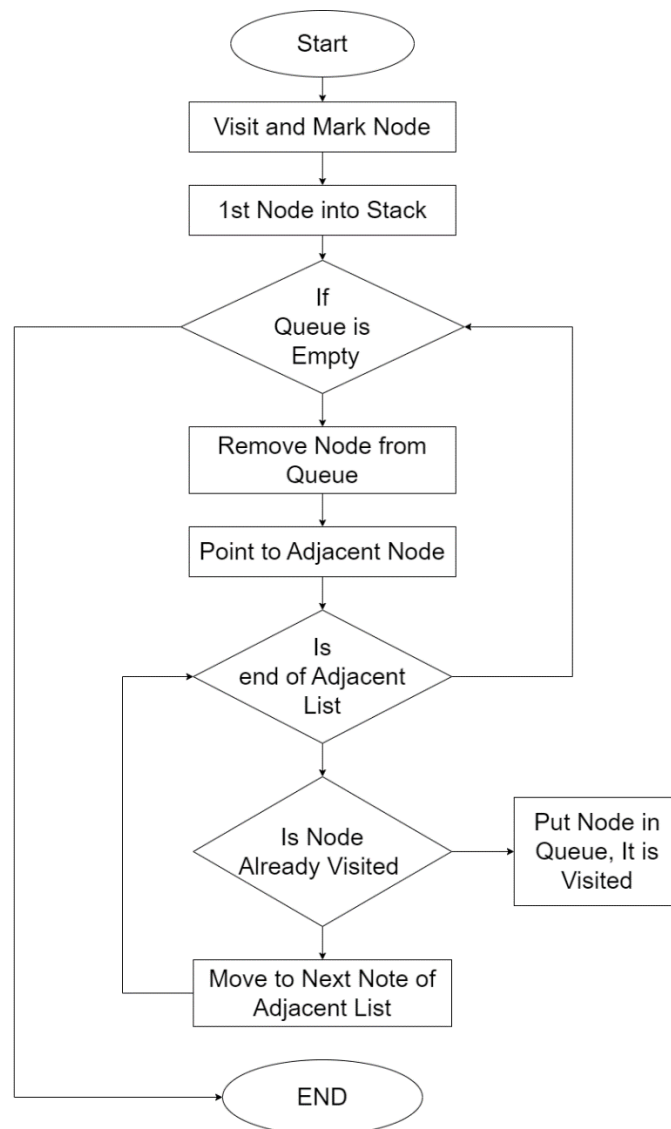
BFS starts from a node, then it checks all the nodes at distance one from the beginning node, then it checks all the nodes at distance two, and so on. So as to recollect the nodes to be visited, BFS uses a queue.

The steps of the algorithm work as follow:

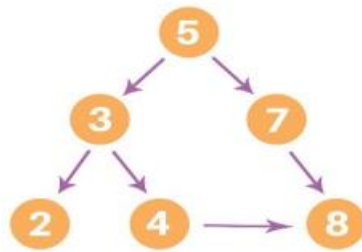
1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

Many times, a graph may contain two different disconnected parts and therefore to make sure that we have visited every vertex, we can also run the BFS algorithm at every node.

BFS Flowchart:



Example:



```
#defining function

def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        m = queue.pop(0)
        print(m, end=" ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

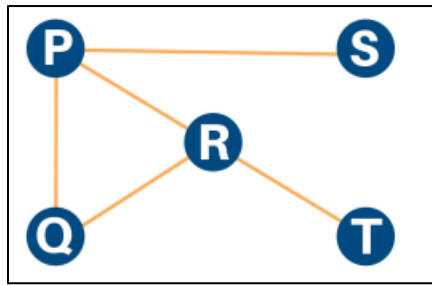
#initializing graph & nodes
graph = {
    'A' : ['B', 'C', 'D'],
    'B' : [],
    'C' : ['D'],
    'D' : ['E'],
    'E' : []
}

visited = [] #initialied visited
queue = [] #initialed queue

#calling bfs method
print("Following is the Breadth first search")
bfs(visited, graph, 'A')
```

Conclusion:

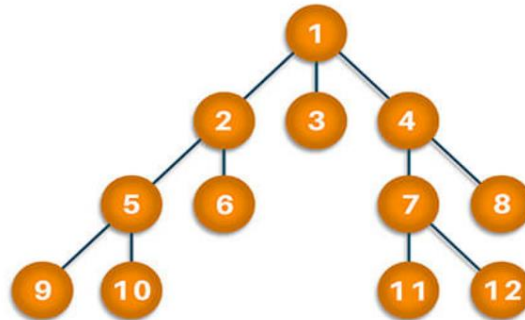
Lab Exercise:



Practical 2

Aim: Write a Program to Implement Depth FirstSearch using Python

Theory:



What do we do once have to solve a maze? We tend to take a route, keep going until we discover a dead end. When touching the dead end, we again come back and keep coming back till we see a path we didn't attempt before. Take that new route. Once more keep going until we discover a dead end. Take a come back again... This is exactly how Depth-First Search works.

The Depth-First Search is a recursive algorithm that uses the concept of backtracking. It involves thorough searches of all the nodes by going ahead if potential, else by backtracking. Here, the word backtrack means once you are moving forward and there are not any more nodes along the present path, you progress backward on an equivalent path to seek out nodes to traverse. All the nodes are progressing to be visited on the current path until all the unvisited nodes are traversed after which subsequent paths are going to be selected.

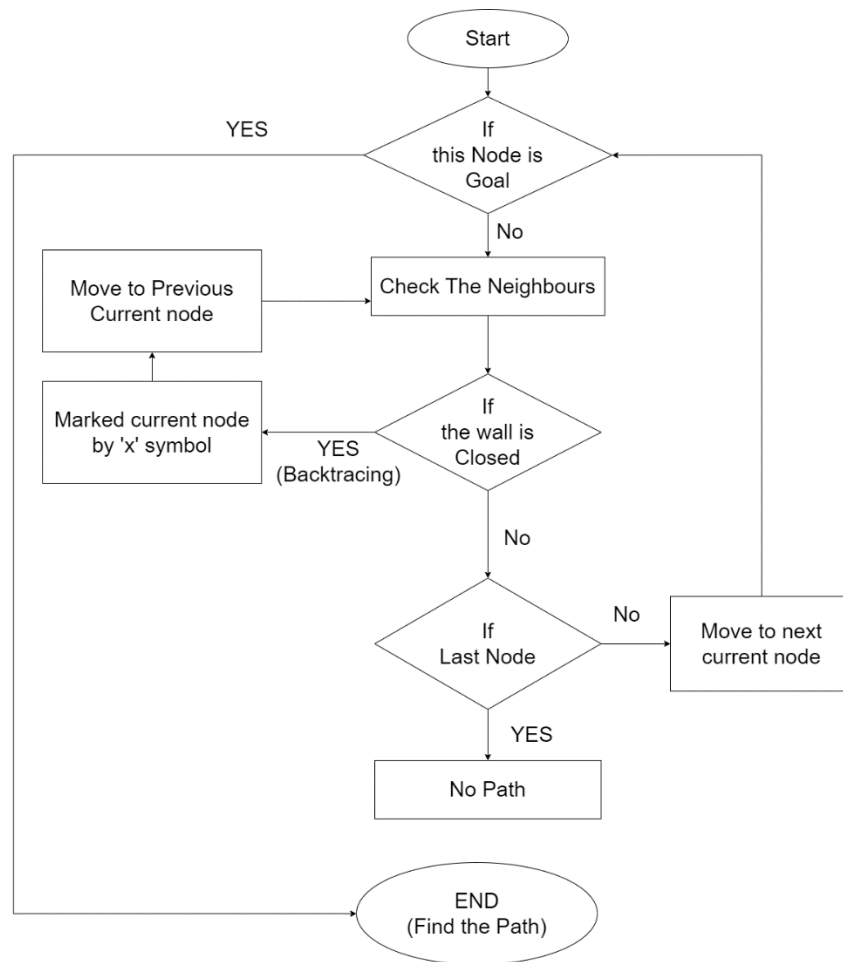
DFS Algorithm:

Before learning the python code for Depth-First and its output, let us go through the algorithm it follows for the same. The recursive method of the Depth-First Search algorithm is implemented using stack. A standard Depth-First Search implementation puts every vertex of the graph into one in all 2 categories: 1) Visited 2) Not Visited. The only purpose of this algorithm is to visit all the vertex of the graph avoiding cycles.

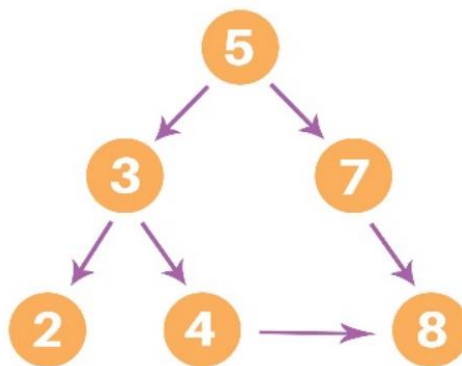
The DSF algorithm follows as:

1. We will start by putting any one of the graph's vertex on top of the stack.
2. After that take the top item of the stack and add it to the visited list of the vertex.
3. Next, create a list of that adjacent node of the vertex. Add the ones which aren't in the visited list of vertexes to the top of the stack.
4. Lastly, keep repeating steps 2 and 3 until the stack is empty.

DFS Flowchart:



Example:



```
# Using a Python dictionary to act as an adjacency list
graph = {
    '5' : ['3', '7'],
    '3' : ['2', '4'],
    '7' : ['8'],
    '2' : [],
    '4' : ['8'],
    '8' : []
}

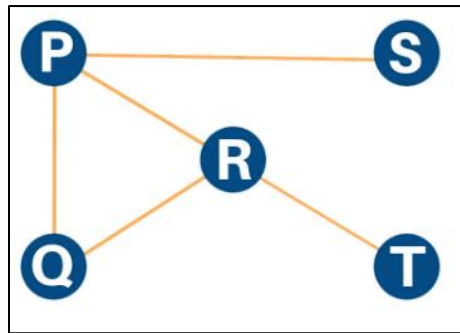
visited = set() # Set to keep track of visited nodes of graph.

def dfs(visited, graph, node): #function for dfs
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, '5')
```

Conclusion:

Lab Exercise:



Practical 3

Aim: Write a Program to Implement Tic-Tac-Toegame using Python

Theory:

We all know about the Tic-Tac-Toe game we use to play in boring lectures, Let us see algorithm to play this game using Python:

This algorithm will help you to write code in any programming language of your choice. Let's see how it's done.

- Create a board using a 2-dimensional array and initialize each element as empty.
 - You can represent empty using any symbol you like. Here, we are going to use a hyphen. ('_')
- Write a function to check whether the board is filled or not.
 - Iterate over the board and return false if the board contains an empty sign or else return true.
- Write a function to check whether a player has won or not.
 - We have to check all the possibilities that we discussed in the previous section.
 - Check for all the rows, columns, and two diagonals.
- Write a function to show the board as we will show the board multiple times to the users while they are playing.
- Write a function to start the game.
 - Select the first turn of the player randomly.
 - Write an infinite loop that breaks when the game is over (either win or draw).
 - Show the board to the user to select the spot for the next move.
 - Ask the user to enter the row and column number.
 - Update the spot with the respective player sign.
 - Check whether the current player won the game or not.
 - If the current player won the game, then print a winning message and break the infinite loop.
 - Next, check whether the board is filled or not.
 - If the board is filled, then print the draw message and break the infinite loop.
- Finally, show the user the final view of the board.

Source Code in Python:

```
import random

class TicTacToe:

    def __init__(self):
        self.board = []

    def create_board(self):
        for i in range(3):
            row = []
            for j in range(3):
                row.append('-')
            self.board.append(row)

    def get_random_first_player(self):
        return random.randint(0, 1)

    def fix_spot(self, row, col, player):
        self.board[row][col] = player

    def is_player_win(self, player):
        win = None

        n = len(self.board)

        # checking rows
        for i in range(n):
            win = True
            for j in range(n):
                if self.board[i][j] != player:
                    win = False
                    break
            if win:
                return win

        # checking columns
        for i in range(n):
            win = True
            for j in range(n):
                if self.board[j][i] != player:
                    win = False
                    break
            if win:
                return win
```

```

# checking diagonals
win = True
for i in range(n):
    if self.board[i][i] != player:
        win = False
        break
if win:
    return win

win = True
for i in range(n):
    if self.board[i][n - 1 - i] != player:
        win = False
        break
if win:
    return win
return False

for row in self.board:
    for item in row:
        if item == '-':
            return False
return True

def is_board_filled(self):
    for row in self.board:
        for item in row:
            if item == '-':
                return False
    return True

def swap_player_turn(self, player):
    return 'X' if player == 'O' else 'O'

def show_board(self):
    for row in self.board:
        for item in row:
            print(item, end=" ")
        print()

def start(self):
    self.create_board()

    player = 'X' if self.get_random_first_player() == 1 else 'O'
    while True:

```

```
print(f'Player {player} turn')

self.show_board()

# taking user input
row, col = list(
    map(int, input("Enter row and column numbers to fix spot: ").split()))
print()

# fixing the spot
self.fix_spot(row - 1, col - 1, player)

# checking whether current player is won or not
if self.is_player_win(player):
    print(f'Player {player} wins the game!')
    break

# checking whether the game is draw or not
if self.is_board_filled():
    print("Match Draw!")
    break

# swapping the turn
player = self.swap_player_turn(player)

# showing the final view of board
print()
self.show_board()

# starting the game
tic_tac_toe = TicTacToe()
tic_tac_toe.start()
```

Output:

Conclusion:

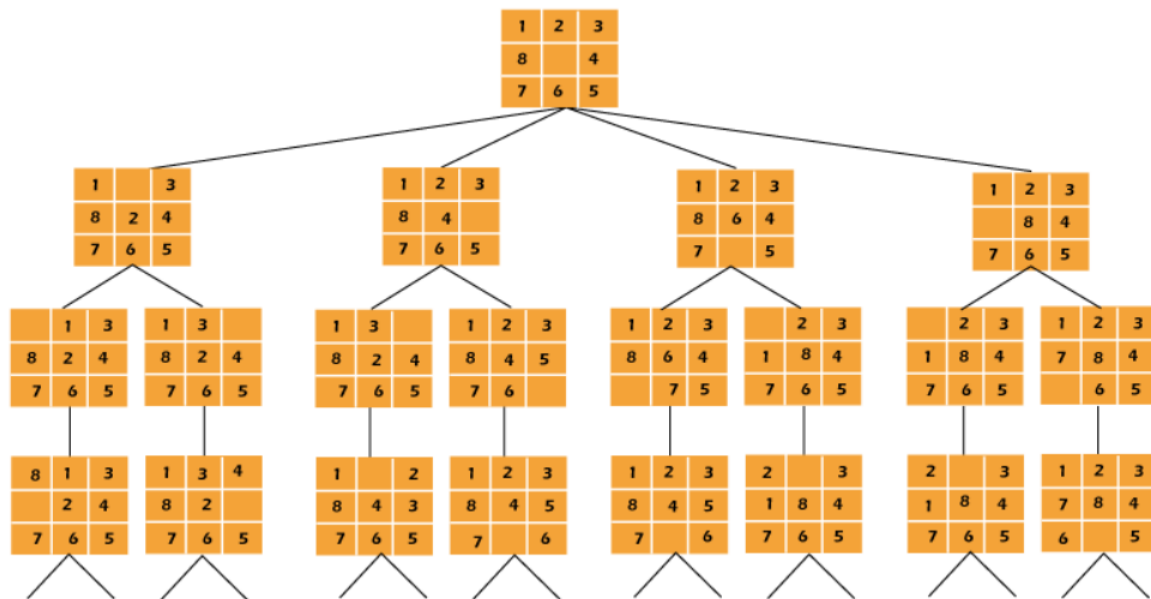
Practical 4

Aim: Write a Program to Implement 8-Puzzleproblem using Python

Theory:

The 8-puzzle problem solution is covered in this article. A 3 by 3 board with 8 tiles (each tile has a number from 1 to 8) and a single empty space is provided. The goal is to use the vacant space to arrange the numbers on the tiles such that they match the final arrangement. Four neighbouring (left, right, above, and below) tiles can be moved into the available area.

For instance,



1. DFS (Brute - Force):

On the state-space tree (Set of all configurations of a particular issue, i.e., all states that may be reached from the beginning state), we can do a depth-first search.

Initial Configuration			Final Configuration		
1	2	3	1	2	3
5	6		5	8	6
7	8	4		7	4

In this solution, further movements might not always send us closer to the objective, but rather further away. Regardless of the initial state, the state-space tree searches down the leftmost route from the root. With this method, an answer node might never be discovered.

2. BFS (Brute - Force) :

We can search the state space tree using a breadth-first approach. It always locates the goal state that is closest to the root. However, the algorithm tries the same series of movements as DFS regardless of the initial state.

3. Branch and Bound :

By avoiding searching in sub-trees which do not include an answer node, an "intelligent" ranking function, also known as an approximation costs function, may frequently speed up the search for an answer node. However, instead of using the backtracking method, it does a BFS-style search.

Basically, Branch and Bound involves three different kinds of nodes.

A live node is a created node whose children have not yet been formed.

The offspring of the E-node which is a live node, are now being investigated. Or to put it another way, an E-node is a node that is currently expanding.

A created node which is not to be developed or examined further is referred to as a dead node. A dead node has already extended all of its offspring.

Costs function :

In the search tree, each node Y has a corresponding costs. The next E-node may be found using the costs function. The E-node with the lowest costs is the next one. The function can be defined as :

$$C(Y) = g(Y) + h(Y)$$

where

$g(Y)$ = the costs of reaching to the current node
from the root.

$h(Y)$ = the costs of reaching to an answer node from the Y.

We suppose that it will cost one unit to move a tile in any direction. In light of this, we create the following costs function for the 8-puzzle algorithm :

$$c(y) = f(y) + h(y)$$

where

$f(y)$ = the path's total length from the root y.

and

$h(y)$ = the amount of the non-blank tiles which are not in
their final goal position (misplaced tiles).

To change state y into a desired state, there are at least $h(y)$ movements required.

Final Algorithm:

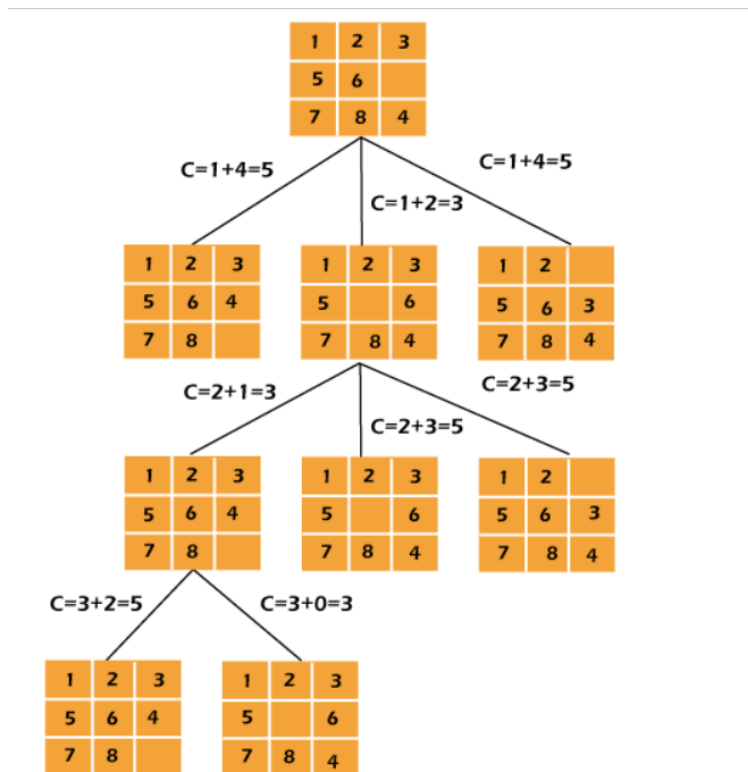
In order to maintain the list of live nodes, algorithm LCSearch employs the functions Least() and Add().

Least() identifies a live node with the least $c(y)$, removes it from the list, and returns it.

Add(y) adds y to the list of live nodes.

Add(y) implements the list of live nodes as a min-heap.

The route taken by the aforementioned algorithm to arrive at the final configuration of the 8-Puzzle from the starting configuration supplied is shown in the diagram below. Keep in mind that only nodes with the lowest costs function value are extended.



1. # Python code to display the way from the root
2. # node to the final destination node for $N \times N - 1$ puzzle
3. # algorithm by the help of Branch and Bound technique
4. # The answer assumes that the instance of the
5. # puzzle can be solved
- 6.
7. # Importing the 'copy' for deepcopy method

```
8. import copy
9.
10. # Importing the heap methods from the python
11. # library for the Priority Queue
12. from heapq import heappush, heappop
13.
14. # This particular var can be changed to transform
15. # the program from 8 puzzle(n=3) into 15
16. # puzzle(n=4) and so on ...
17. n = 3
18.
19. # bottom, left, top, right
20. rows = [ 1, 0, -1, 0 ]
21. cols = [ 0, -1, 0, 1 ]
22.
23. # creating a class for the Priority Queue
24. class priorityQueue:
25.
26.     # Constructor for initializing a
27.     # Priority Queue
28.     def __init__(self):
29.         self.heap = []
30.
31.     # Inserting a new key 'key'
32.     def push(self, key):
33.         heappush(self.heap, key)
34.
35.     # funct to remove the element that is minimum,
36.     # from the Priority Queue
37.     def pop(self):
38.         return heappop(self.heap)
39.
40.     # funct to check if the Queue is empty or not
41.     def empty(self):
```

```
42.     if not self.heap:
43.         return True
44.     else:
45.         return False
46.
47. # structure of the node
48. class nodes:
49.
50.     def __init__(self, parent, mats, empty_tile_posi,
51.                 costs, levels):
52.
53.         # This will store the parent node to the
54.         # current node And helps in tracing the
55.         # path when the solution is visible
56.         self.parent = parent
57.
58.         # Useful for Storing the matrix
59.         self.mats = mats
60.
61.         # useful for Storing the position where the
62.         # empty space tile is already existing in the matrix
63.         self.empty_tile_posi = empty_tile_posi
64.
65.         # Store no. of misplaced tiles
66.         self.costs = costs
67.
68.         # Store no. of moves so far
69.         self.levels = levels
70.
71.         # This func is used in order to form the
72.         # priority queue based on
73.         # the costs var of objects
74.     def __lt__(self, nxt):
75.         return self.costs < nxt.costs
```

```

76.
77. # method to calc. the no. of
78. # misplaced tiles, that is the no. of non-blank
79. # tiles not in their final posi
80. def calculateCosts(mats, final) -> int:
81.
82.     count = 0
83.     for i in range(n):
84.         for j in range(n):
85.             if ((mats[i][j]) and
86.                 (mats[i][j] != final[i][j])):
87.                 count += 1
88.
89.     return count
90.
91. def newNodes(mats, empty_tile_posi, new_empty_tile_posi,
92.              levels, parent, final) -> nodes:
93.
94.     # Copying data from the parent matrixes to the present matrixes
95.     new_mats = copy.deepcopy(mats)
96.
97.     # Moving the tile by 1 position
98.     x1 = empty_tile_posi[0]
99.     y1 = empty_tile_posi[1]
100.     x2 = new_empty_tile_posi[0]
101.     y2 = new_empty_tile_posi[1]
102.     new_mats[x1][y1], new_mats[x2][y2] = new_mats[x2][y2], new_mats[x1]
    [y1]
103.
104.     # Setting the no. of misplaced tiles
105.     costs = calculateCosts(new_mats, final)
106.
107.     new_nodes = nodes(parent, new_mats, new_empty_tile_posi,
108.                       costs, levels)

```

```

109.         return new_nodes
110.
111.     # func to print the N by N matrix
112.     def printMatsrix(mats):
113.
114.         for i in range(n):
115.             for j in range(n):
116.                 print("%d " % (mats[i][j]), end = " ")
117.
118.             print()
119.
120.     # func to know if (x, y) is a valid or invalid
121.     # matrix coordinates
122.     def isSafe(x, y):
123.
124.         return x >= 0 and x < n and y >= 0 and y < n
125.
126.     # Printing the path from the root node to the final node
127.     def printPath(root):
128.
129.         if root == None:
130.             return
131.
132.         printPath(root.parent)
133.         printMatsrix(root.mats)
134.         print()
135.
136.     # method for solving N*N - 1 puzzle algo
137.     # by utilizing the Branch and Bound technique. empty_tile_posi is
138.     # the blank tile position initially.
139.     def solve(initial, empty_tile_posi, final):
140.
141.         # Creating a priority queue for storing the live
142.         # nodes of the search tree

```

```

143.     pq = priorityQueue()
144.
145.     # Creating the root node
146.     costs = calculateCosts(initial, final)
147.     root = nodes(None, initial,
148.                 empty_tile_posi, costs, 0)
149.
150.     # Adding root to the list of live nodes
151.     pq.push(root)
152.
153.     # Discovering a live node with min. costs,
154.     # and adding its children to the list of live
155.     # nodes and finally deleting it from
156.     # the list.
157.     while not pq.empty():
158.
159.         # Finding a live node with min. estimatsed
160.         # costs and deleting it form the list of the
161.         # live nodes
162.         minimum = pq.pop()
163.
164.         # If the min. is ans node
165.         if minimum.costs == 0:
166.
167.             # Printing the path from the root to
168.             # destination;
169.             printPath(minimum)
170.             return
171.
172.         # Generating all feasible children
173.         for i in range(n):
174.             new_tile_posi = [
175.                 minimum.empty_tile_posi[0] + rows[i],
176.                 minimum.empty_tile_posi[1] + cols[i], ]

```

```
177.
178.         if isSafe(new_tile_posi[0], new_tile_posi[1]):
179.
180.             # Creating a child node
181.             child = newNodes(minimum.mats,
182.                             minimum.empty_tile_posi,
183.                             new_tile_posi,
184.                             minimum.levels + 1,
185.                             minimum, final,)
186.
187.             # Adding the child to the list of live nodes
188.             pq.push(child)
189.
190.     # Main Code
191.
192.     # Initial configuration
193.     # Value 0 is taken here as an empty space
194.     initial = [ [ 1, 2, 3 ],
195.                [ 5, 6, 0 ],
196.                [ 7, 8, 4 ] ]
197.
198.     # Final configuration that can be solved
199.     # Value 0 is taken as an empty space
200.     final = [ [ 1, 2, 3 ],
201.              [ 5, 8, 6 ],
202.              [ 0, 7, 4 ] ]
203.
204.     # Blank tile coordinates in the
205.     # initial configuration
206.     empty_tile_posi = [ 1, 2 ]
207.
208.     # Method call for solving the puzzle
209.     solve(initial, empty_tile_posi, final)
```

OutPut:

Conclusion:

Practical 5

Aim: Prepare an application based on Data Structures and Algorithms for Creating the Bill of Grocery store.

Introduction: The practical assignment aims to develop an application that utilizes data structures and algorithms to create a bill for a grocery store. The application will be implemented using Python programming language. This practical will help you understand the application of data structures and algorithms in real-world scenarios.

Requirements: To complete this practical, you will need the following:

Python programming environment (e.g., Anaconda, Jupyter Notebook, or any IDE of your choice)

1. Basic understanding of Python programming
2. Familiarity with data structures such as lists and dictionaries
3. Knowledge of algorithms, especially searching and sorting algorithms
4. Sample dataset of grocery items (can be obtained from an external source or created manually)

Procedure:

1. Data Structures:

- Begin by defining the data structures required for the application. In this case, you will need a data structure to store information about the grocery items and their corresponding prices. A dictionary is an appropriate choice for this purpose, where the grocery items are the keys, and the prices are the values.

2. Displaying Available Items:

- Implement a function to display the available grocery items to the user. Iterate over the dictionary and print each item along with its price.

3. Creating the Bill:

- Implement a function to create the bill based on the items selected by the user. Prompt the user to enter the items they want to purchase and their quantities. Calculate the total cost and display the bill.

4. Sorting the Bill:

- Implement an algorithm to sort the bill in alphabetical order by item name. This will make the bill easier to read.

5. Main Program:

- Create a main function that calls the above-defined functions in the appropriate order.

Python Source Code:

```
from tabulate import tabulate

grocery_items = {
    'apple': 2.99,
    'banana': 1.49,
    'milk': 3.99,
    'bread': 2.49,
    # Add more grocery items and their prices
}

bill = {}

def display_items():
    print("Available items:")
    for item, price in grocery_items.items():
        print(f'{item}: ${price:.2f}')

def create_bill():
    total_cost = 0
    global bill # Make the 'bill' dictionary global

    while True:
        item = input("Enter item name (or 'q' to quit): ")
        if item == 'q':
            break

        if item in grocery_items:
            quantity = int(input("Enter quantity: "))
            price = grocery_items[item]
            cost = price * quantity
            bill[item] = {'quantity': quantity, 'cost': cost}
            total_cost += cost
        else:
            print("Invalid item. Please try again.")

    print("\nBill:")
    print(tabulate([(item, details['quantity'], f"${grocery_items[item]:.2f}",
f"${details['cost']:.2f}") for item, details in bill.items()], headers=["Item", "Quantity", "Unit
Price", "Total Cost"])))
```

```
print(f"\nTotal cost: ${total_cost:.2f}")

def sort_bill():
    sorted_bill = sorted(bill.items(), key=lambda x: x[0])
    print("\nSorted Bill:")
    print(tabulate([(item, details['quantity'], f"${grocery_items[item]:.2f}",
f"${details['cost']:.2f}") for item, details in sorted_bill], headers=["Item", "Quantity", "Unit
Price", "Total Cost"])))

def main():
    display_items()
    create_bill()
    sort_bill()

if __name__ == '__main__':
    main()
```

Output:

Conclusion:

Lab Exercise

Q. Write a Program to note the attendance of a group of students during a particular course for each Session and at the end group the students in 4 groups:

Group 1: Students having attendance 91-100%

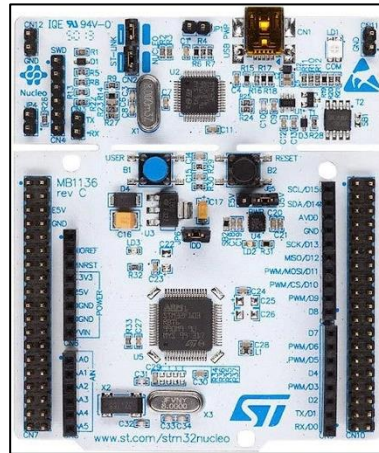
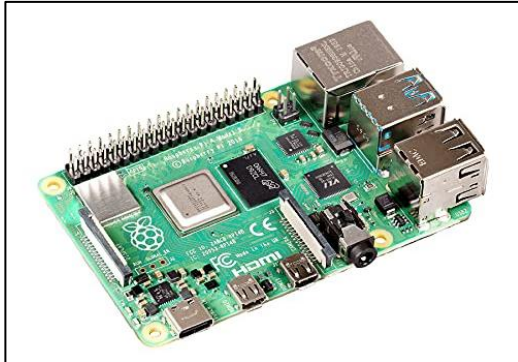
Group 2: Students having attendance 75-90%

Group 3: Students having attendance 60-75%

Group 4: Students having Attendance below 60%

Practical 6

Aim: Introduction to Micro Controllers and Embedded World



Objective:

The objective of this practical session is to introduce students to microcontrollers and familiarize them with three popular development platforms: Arduino, Raspberry Pi, and STM32. Students will learn about the features, capabilities, and applications of these platforms, as well as gain hands-on experience with basic programming and interfacing.

Table of Contents:

1. Introduction to Microcontrollers
2. Overview of Embedded Systems
3. Introduction to Raspberry Pi
4. Getting Started with Raspberry Pi
5. Raspberry Pi 4 - Features and Specifications
6. Interfacing with Raspberry Pi
7. Introduction to Arduino Uno
8. Introduction to STM32 Nucleo Board
9. Comparison of Microcontrollers

1. Introduction to Microcontrollers:

- Definition and purpose of microcontrollers
- Role of microcontrollers in embedded systems
- Importance of microcontrollers in Robotics and Artificial Intelligence

2. Overview of Embedded Systems:

- Definition and characteristics of embedded systems
- Components and architecture of embedded systems
- Applications of embedded systems in various industries

3. Introduction to Raspberry Pi:

- Introduction to Raspberry Pi and its history
- Raspberry Pi models and their differences
- Key features and capabilities of Raspberry Pi boards

4. Getting Started with Raspberry Pi:

- Raspberry Pi setup and configuration
- Connecting peripherals (keyboard, mouse, display, etc.)
- Installing the operating system (e.g., Raspbian)

5. Raspberry Pi 4 - Features and Specifications:

- Overview of Raspberry Pi 4 board
- Detailed explanation of the hardware components
- Specifications, including CPU, RAM, storage options, etc.

6. Interfacing with Raspberry Pi:

- GPIO (General-Purpose Input/Output) pins on Raspberry Pi
- Connecting sensors, actuators, and other devices
- Programming GPIO using Python

7. Introduction to Arduino Uno:

- Overview of Arduino Uno board
- Key features and capabilities

- Programming Arduino using the Arduino IDE

8. Introduction to STM32 Nucleo Board:

- Overview of STM32 Nucleo board
- Features and advantages of STM32 microcontrollers
- Programming STM32 Nucleo using the STM32CubeIDE

9. Comparison of Microcontrollers:

- Comparative analysis of Raspberry Pi, Arduino Uno, and STM32 Nucleo
- Differentiating factors, such as processing power, I/O capabilities, etc.
- Use cases and suitability for various projects

10. Conclusion:

Practical 7

Aim: Write Program and Prepare Circuit to Perform Voice Recognition Activity using Raspberry Pi 4.

Introduction: The Voice Recognition System using Raspberry Pi 4 is a practical project that aims to introduce students to the fundamentals of AI and its applications in the field of robotics. This manual provides step-by-step instructions for setting up the voice recognition system using a Raspberry Pi 4.

Requirements: To successfully complete the Voice Recognition System project, you will need the following components:

1. Raspberry Pi 4 Model B (4GB or higher recommended)
2. MicroSD card (16GB or higher) with Raspbian OS installed
3. USB microphone or a USB sound card with a built-in microphone
4. Speaker or headphones for audio output
5. Internet connectivity (Wi-Fi or Ethernet)
6. HDMI cable and monitor (optional, for setup and troubleshooting)
7. Power supply for Raspberry Pi 4 (official Raspberry Pi power supply recommended)
8. Keyboard and mouse (USB or wireless) for Raspberry Pi 4

Note: Make sure to have a stable power source and take necessary precautions while handling electronic components.

Setup:

1. Connect the Raspberry Pi 4:
 - Insert the MicroSD card with Raspbian OS into the Raspberry Pi 4.
 - Connect the USB microphone or sound card to one of the USB ports.
 - Connect the speaker or headphones to the audio jack or USB port.
 - Connect the HDMI cable and monitor (optional) for initial setup.
 - Connect the keyboard and mouse to USB ports.
2. Power On:
 - Plug in the power supply to the Raspberry Pi 4.
 - The Raspberry Pi will boot up, and you should see the desktop environment.
3. Configure Raspbian OS:
 - Follow the on-screen instructions to set up the basic configuration of Raspbian, including language, keyboard layout, and Wi-Fi (if applicable).
 - Make sure your Raspberry Pi has internet connectivity to install necessary packages.
4. Install Dependencies:

Open a terminal window and execute the following commands sequentially:

```
sudo apt update
sudo apt upgrade
sudo apt install python3-pip python3-pyaudio python3-pyaudioportaudio flac
sudo pip3 install SpeechRecognition
sudo pip3 install pytsx3
```

5. Testing the Microphone:

- Open a terminal window and execute the following command:

```
arecord -d 5 test.wav
```

- This command records audio for 5 seconds and saves it as "test.wav".

Verify that the audio was recorded correctly by playing it back using the following command:

```
aplay test.wav
```

Adjust the microphone sensitivity or gain if the audio quality is not satisfactory.

6. Coding the Voice Recognition System:

- Create a new Python script using your preferred code editor.

Import the necessary libraries:

```
import speech_recognition as sr
import pyttsx3
```

Initialize the speech recognition and text-to-speech engines:

```
recognizer = sr.Recognizer()
engine = pyttsx3.init()
```

Define a function to listen to the user's voice and convert it to text:

```
def listen():
    with sr.Microphone() as source:
        print("Listening...")
        audio = recognizer.listen(source)
        try:
            text = recognizer.recognize_google(audio)
            print("You said:", text)
            return text
        except sr.UnknownValueError:
            print("Sorry, I could not understand.")
            return ""
        except sr.RequestError as e:
            print("Speech recognition service error:", str(e))
            return ""
```

Define a function to convert text to speech and play it through the speaker:

```
def speak(text):
    engine.say(text)
    engine.runAndWait()
```

```
if __name__ == "__main__":
```

```
try:
    text = listen()
    if text:
        speak("You said: " + text)
except KeyboardInterrupt:
    pass
```

• Call the listen() function to capture voice input and the speak() function to provide the output.

7. Save and Run the Program:

- Save the Python script with a suitable name and the ".py" extension.
- Open a terminal window and navigate to the directory where the script is saved.

Execute the script using the following command:

```
python3 script_name.py
```

Test the voice recognition system by speaking into the microphone and verifying the text output.

Conclusion:

Practical 8

Aim: Write Program and Prepare Circuit to Perform Face Recognition Activity and Take Attendance using Raspberry Pi 4.

1. Introduction: The Face Recognition System using Raspberry Pi 4 is a practical project designed to teach the fundamentals of AI in the context of robotics and artificial intelligence in mechanical engineering. This manual will guide you through the process of building a face recognition system using the Raspberry Pi 4. The system will be able to recognize faces, associate them with names, and determine the total number of people in an image captured by the camera.

2. Requirements: To successfully complete this project, you will need the following components and software:

Hardware:

- Raspberry Pi 4 Model B (4GB or higher RAM recommended)
- Raspberry Pi Camera Module (compatible with Raspberry Pi 4)
- HDMI display and cables (for initial setup)
- USB keyboard and mouse (for initial setup)
- Power supply for Raspberry Pi 4
- MicroSD card (16GB or higher capacity)
- MicroSD card reader

Software:

- Raspbian OS (latest version)
- Python programming language (version 3.7 or higher)
- OpenCV library for Python (version 4.5 or higher)
- NumPy library for Python (version 1.19 or higher)
- Face recognition library for Python (version 1.3.0 or higher)
- Pretrained face recognition model (such as the dlib model)

3. Raspberry Pi 4 Program: Follow the steps below to create the Raspberry Pi 4 program for the face recognition system:

Step 1: Set up Raspberry Pi 4

- Install Raspbian OS on the microSD card and insert it into the Raspberry Pi 4.
- Connect the HDMI display, USB keyboard, and mouse to the Raspberry Pi 4.

- Power on the Raspberry Pi 4 and complete the initial setup process.

Step 2: Install Required Software

Open a terminal on the Raspberry Pi 4 and update the system:

1. `sudo apt-get update`
2. `sudo apt-get upgrade`

Install Python 3 and pip:

```
sudo apt-get install python3 python3-pip
```

Install OpenCV, NumPy, and face recognition libraries:

```
pip3 install opencv-python numpy face-recognition
```

Download the pretrained face recognition model:

```
wget http://dlib.net/files/dlib_face_recognition_resnet_model_v1.dat.bz2  
bzip2 -dk dlib_face_recognition_resnet_model_v1.dat.bz2
```

Step 3: Write the Face Recognition Program

- Use a text editor (such as Nano) to create a new Python file, e.g., `face_recognition.py`.

Import the necessary libraries and functions:

```
import cv2  
import face_recognition
```

Initialize the camera:

```
video_capture = cv2.VideoCapture(0)
```

Load the known faces and their names:

```
known_faces = []  
known_names = []
```

Enter a loop to continuously capture frames from the camera:

```
while True:  
    ret, frame = video_capture.read()  
    # Perform face detection and recognition  
    # ...  
    # Display the recognized faces and their names
```

```
# ...  
# Break the loop if a key is pressed (e.g., 'q')  
# ...
```

- • Add code to perform face detection and recognition using the `face_recognition` library.
- • Display the recognized faces and their names on the frame using `OpenCV`.
- • Break the loop if a specific key (e.g., 'q') is pressed.
- Step 4: Run the Program
- • Save the `face_recognition.py` file and exit the text editor.
- • In the terminal, navigate to the directory where the file is saved.
- Run the program using Python:

```
python3 face_recognition.py
```

Conclusion:

Practical 9

Aim: Prepare an Obstacle Avoiding Self-Driving Robot Using Raspberry Pi 4.

Introduction: The practical subject "Obstacle Avoiding Self-Driving Robot using Raspberry Pi 4" is designed to provide students with hands-on experience in building and programming a self-driving robot that can navigate its environment while avoiding obstacles. The project utilizes the Raspberry Pi 4, a versatile single-board computer, to control the robot's movements and implement artificial intelligence algorithms for obstacle detection and avoidance.

Requirements:

1. Raspberry Pi 4: The Raspberry Pi 4 is the main controller board for the self-driving robot. Ensure you have a working Raspberry Pi 4 with necessary peripherals such as a power supply, microSD card, keyboard, mouse, and display.
2. Robot Chassis: Select a suitable robot chassis that can accommodate the Raspberry Pi 4 and the required additional components such as motors, wheels, and sensors. You can either purchase a ready-to-use robot kit or assemble one yourself using individual components.
3. Motor Driver: A motor driver circuit is required to control the robot's motors. Choose a motor driver compatible with the Raspberry Pi 4, such as an L298N motor driver module.
4. DC Motors: Acquire two DC motors suitable for your robot chassis. The motors should have enough torque to drive the robot and provide smooth movement.
5. Wheels: Choose appropriate wheels that fit your robot chassis and motors. The wheels should have good traction for effective movement.
6. Ultrasonic Sensors: Ultrasonic sensors are used for obstacle detection. Obtain at least two ultrasonic sensors for the front of the robot. These sensors will help the robot measure distances and detect obstacles in its path.
7. Jumper Wires: Ensure you have an adequate supply of jumper wires to establish connections between the Raspberry Pi 4, motor driver, sensors, and other components.
8. Power Bank or Battery Pack: To make the self-driving robot portable, you will need a power bank or battery pack capable of providing sufficient power to run the Raspberry Pi 4 and motors.

Raspberry Pi 4 Program: To program the self-driving robot using Raspberry Pi 4, you can use Python programming language and the GPIO (General-Purpose Input/Output) library for controlling the GPIO pins of the Raspberry Pi. Here is a sample program outline:

Import the necessary libraries:

```
import RPi.GPIO as GPIO
import time
```

Set up the GPIO pins:

```
GPIO.setmode(GPIO.BCM)

# Define motor control pins
motor1_pin1 = 18
motor1_pin2 = 23
motor2_pin1 = 24
```

```
motor2_pin2 = 25

# Set ultrasonic sensor pins
trigger_pin = 20
echo_pin = 21

# Set GPIO pin modes
GPIO.setup(motor1_pin1, GPIO.OUT)
GPIO.setup(motor1_pin2, GPIO.OUT)
GPIO.setup(motor2_pin1, GPIO.OUT)
GPIO.setup(motor2_pin2, GPIO.OUT)
GPIO.setup(trigger_pin, GPIO.OUT)
GPIO.setup(echo_pin, GPIO.IN)
```

Define functions for controlling the motors:

```
def forward():
    # Code to move the robot forward

def backward():
    # Code to move the robot backward

def left():
    # Code to turn the robot left

def right():
    # Code to turn the robot right

def stop():
    # Code to stop the robot
```

Implement obstacle detection and avoidance:

```
def measure_distance():
    # Code to measure the distance using the ultrasonic sensor

def avoid_obstacle():
    # Code to avoid obstacles by turning the robot
```

Create the main program loop:

```
while True:
    distance = measure_distance()

    if distance > threshold:
        forward()
    else:
        avoid_obstacle()
```

Conclusion:

Practical 9.1

Aim: Self-Driving Robot with RFID Sensors and Camera using Raspberry Pi 4

Requirements:

1. Raspberry Pi 4 Model B (4GB or higher)0
2. MicroSD card (16GB or higher) with Raspbian OS installed
3. Power supply for Raspberry Pi 4
4. Motor driver board (compatible with Raspberry Pi)
5. DC motors (2)
6. Wheels (2)
7. RFID sensors (compatible with Raspberry Pi)
8. Camera module (compatible with Raspberry Pi)
9. Jumper wires
10. Breadboard (optional, for prototyping)
11. Chassis for the robot (can be purchased or built using a DIY approach)
12. USB keyboard and mouse0
13. HDMI monitor or TV
14. Internet connectivity (Ethernet cable or Wi-Fi)

Program:

The following program utilizes Python programming language to control the self-driving robot using RFID sensors and a camera. The program should be run on the Raspberry Pi 4.

Set up the Raspberry Pi 4:

- a. Insert the microSD card with Raspbian OS into the Raspberry Pi 4.
- b. Connect the power supply to the Raspberry Pi 4.
- c. Connect the HDMI monitor or TV, keyboard, and mouse to the Raspberry Pi 4.
- d. Establish an internet connection (Ethernet cable or Wi-Fi).

Install necessary libraries:

- a. Open a terminal window on the Raspberry Pi 4.
- b. Update the package list:

```
sudo apt update
```

c. Install the RPi.GPIO library for controlling the GPIO pins:

```
sudo apt install python3-rpi.gpio
```

d. Install the picamera library for camera support:

```
sudo apt install python3-picamera
```

e. Install any other required libraries as per the specific RFID sensor or motor driver board used.

Connect the hardware components:

a. Connect the motor driver board to the appropriate GPIO pins on the Raspberry Pi 4, following the board's documentation.

b. Connect the RFID sensors to the designated GPIO pins on the Raspberry Pi 4, following the sensor's documentation.

c. Connect the camera module to the dedicated camera port on the Raspberry Pi 4.

Write the program:

a. Open a text editor on the Raspberry Pi 4.

b. Write the Python program to control the self-driving robot. Here's a basic example to get started:

```
import RPi.GPIO as GPIO
import time
from picamera import PiCamera

# Define GPIO pins for motor control
motor1_pin1 = 17
motor1_pin2 = 18
motor2_pin1 = 22
motor2_pin2 = 23

# Initialize motor control
GPIO.setmode(GPIO.BCM)
GPIO.setup(motor1_pin1, GPIO.OUT)
GPIO.setup(motor1_pin2, GPIO.OUT)
GPIO.setup(motor2_pin1, GPIO.OUT)
GPIO.setup(motor2_pin2, GPIO.OUT)
```

```
# Initialize camera
camera = PiCamera()

# Main loop
while True:
    # Read RFID sensors
    # Implement logic to detect charging point using RFID sensors

    # If charging point detected:
    # Stop the robot
    # Capture an image using the camera
    # Save the image to a desired location

    # If charging point not detected:
    # Continue moving the robot towards the charging point

    # Implement motor control logic using GPIO pins
    # Use GPIO.output() to set motor pins high or low for controlling the motor direction and speed

    time.sleep(0.1)

# Clean up GPIO pins
GPIO.cleanup()
```

Note: The above code is a basic template. You need to implement the logic for RFID sensor detection and movement towards the charging point based on your specific hardware and requirements.

Save the program:

- a. Save the program file with a ".py" extension (e.g., self_driving_robot.py).

Run the program:

- a. Open a terminal window on the Raspberry Pi 4.
- b. Navigate to the directory where the program file is saved.
- c. Run the program using the following command:

```
python3 self_driving_robot.py
```

Ensure that you have implemented the necessary logic for RFID sensor detection and movement towards the charging point in the program. Additionally, consider adding error handling and appropriate algorithms to ensure the safe and efficient operation of the self-driving robot.

Conclusion: